Problem statement:

Last year, I was building a RAG System from scratch. I was new to the concept of rag and wanted to build it from scratch to understand it better. I was using chatgpt as my coding assistant and for planning the next steps in that project. I built it version-wise, where each version was adding something new to the system improving performance from the previous version.

As I went deeper and deeper in that project by solving my doubts, correcting my code & planning & re-planning steps, the chat got too long and it started to hang.

- This is a Context-window/ attention problem. The model technically still had tokens but retrieval from long-context degraded which resulted in things getting lost or de-prioritized as the Conversation grew.

The second problem was it started to forget what my initial ideas for the project were and what I did earlier in that project. So, I then had to constantly tell it that we had changed from this plan to this, this tool to this, this code to this and this project consumed extra time,extra effort to build which was very frustrating in the end.

- This problem occurred because there was no durable, structured record of decisions made and superseded that the model can reliably consult, independent of raw conversation length.

Affected users:

This problem is not unique to my project. It affects anyone working on long-running, iterative tasks with an AI assistant, for example:

- Software developers building large applications over weeks or months (feature development, debugging, architecture changes)
- Writers or authors developing long-form content such as books, reports, or documentation where previous plot points or design decisions must remain consistent.

Estimated impact:

Re-explaining previous decisions, reconstructing project history, and correcting outdated suggestions likely increased the total project time by 20–30%. On longer projects spanning several weeks or months, this can easily translate to 10–30 additional hours spent recovering context instead of making forward progress.

Constraints:

Constraint 1 - Correctness/Privacy:

I would specifically focus on correctness rather than privacy for this project. As the concrete failure case we identified earlier, We first decided to store embeddings and chunks together in vector_store.pkl. Later, we explicitly migrated to a FAISS index with chunks stored separately. Both decisions remain semantically relevant, but only the second one represents the current architecture. If a memory system retrieves both memories based only on similarity, it may give the model two conflicting but individually valid facts. The problem is not retrieval failure. Both memories are relevant.
The problem is that the system has no reliable way to know that a new decision should supersede the old decision. It is a real memory-system constraint because a memory system is not just trying to retrieve information, It is trying to retrieve the currently valid information. That requires more than semantic similarity. It requires things like Temporal ordering & version awareness.

Privacy: Privacy is a separate concern that I am explicitly not covering in depth in the baseline implementation. The current project does not involve real user PII or a production multi-user environment, so the primary focus is on whether the system can retrieve the correct and currently valid memory. A production memory system that stores personal facts would additionally need mechanisms for user consent, data ownership, retention and deletion policies, and controls over what information is allowed to be remembered.

Constraint 2 - Latency:

Latency is a real constraint because I have already seen the retrieval path become expensive in my own RAG project. In the early version, every query embedded the question and then compared it against every stored document embedding using a Python loop. That was fine for a small document, but the work grows with the number of stored chunks. I later replaced that brute-force search with FAISS, which made the

retrieval layer more scalable. A memory system has the same problem: if every new message has to search an ever-growing history using an increasingly expensive retrieval path, the assistant gradually becomes slower even when the user asks a simple question.

Constraint 3 - Token budget:

Token budget is a real constraint because the memory system has to inject retrieved memories into the same finite context window that already contains the system prompt, the current conversation, the user's latest message, and space for the model's response.

This is directly related to the RAG system I built: even when retrieval returns useful chunks, sending too many of them to the LLM would eventually crowd the prompt with context that may be individually relevant but collectively unnecessary. The same problem becomes more difficult in a memory system because a single question about my RAG project might retrieve the original `vector_store.pkl` decision, the later FAISS migration, the FastAPI changes, the confidence-gating decision, and other project history.

If the system has no memory budget, it can inject all of these into the prompt, consuming tokens that should be used for the current conversation and answer. The failure is not necessarily that the system cannot retrieve the right memories; it is that too many retrieved memories compete for a finite context window, potentially pushing out important recent context or leaving insufficient space for the model to reason and generate a useful answer. Therefore, the memory system needs a fixed per-request budget for how much historical memory it is allowed to inject, with selection and compression happening before context injection.

Constraint 4 - Cost:

Cost is a different constraint because it is not about whether one request can fit inside the context window; it is about the cumulative work performed across every message and every user over time.

In my RAG system, the pipeline already performs work for each query: it creates an embedding, searches the FAISS index, evaluates retrieval confidence, and, when allowed, calls the LLM. A memory system adds another layer of potentially recurring work around every conversational turn: extracting candidate memories, deciding whether a message contains anything worth storing, embedding new memories,

searching existing memories, and potentially using an LLM for memory extraction or conflict resolution.

This cost can appear even on messages such as “okay,” “thanks,” or a simple follow-up that creates no durable information. At small scale, local models hide much of this cost because the RAG project runs locally, but at scale, for example with 100,000 messages per day across many users, repeatedly running embedding and memory-processing operations on every turn can become a significant infrastructure and inference expense. If cost is ignored, the system may technically work but become economically impractical because it is paying to process large amounts of conversational traffic that produce little or no useful memory.

Constraint 5 - Multi-user isolation:

Multi-user isolation is a fundamental constraint because the entire purpose of the memory system is to retrieve information and inject it into a model's context, so a retrieval-boundary mistake can directly become a privacy breach.

My RAG project currently has one local FAISS index containing the chunks for one knowledge base, but a multi-user memory system would contain memories belonging to many different users or tenants. If those memories are placed in a shared index without reliable user or tenant filtering, a query from User B could retrieve a semantically similar memory created from User A's conversation. For example, if User A had discussed a private project architecture, financial information, or personal decision, and User B later asked a semantically similar question, the retrieval layer could return User A's memory and inject it into User B's prompt. The model might then confidently use that memory when generating User B's answer.

The worst-case failure is therefore not simply “the assistant gives the wrong answer”; it is that one user's private historical information is exposed to another user because the memory retrieval layer crossed the ownership boundary. In a system whose core function is context injection, isolation must therefore be enforced before retrieval results reach the LLM, through mechanisms such as user/tenant-scoped storage or mandatory filtering, rather than relying on the model to decide which retrieved memories it should ignore.

Open Questions:

1) **Privacy:** We have deliberately scoped privacy out of the baseline implementation, but a production memory system would still need to decide what information should be considered sensitive enough to not store in the first place. Should the system rely on a fixed definition of PII, or should the user/system be able to decide what counts as private and what can be remembered?

   This is a real tradeoff between remembering more useful information and reducing the risk of storing something that should never have become a memory.

2) **Summarization**: For Approach 1, we identified summarizing old conversations as a way to avoid losing important context, but we haven't decided what should survive the summarization and what can be dropped.

   A more detailed summary preserves more project history but consumes more tokens and may still become too large over time, A shorter summary is cheaper and easier to use but risks losing the exact details behind important decisions.

3) **Token budget:** We established that memory cannot be allowed to consume the entire context window, but we haven't decided how much of the available context should be allocated to retrieved memories versus the live conversation and the model's response. Giving memory more space may help recover older context, but it leaves less room for the current conversation and response, giving memory less space keeps the interaction focused but increases the chance of missing important historical context.

4) **Conflict resolution**: We know semantic similarity alone cannot tell us whether two similar memories are both valid or whether one has replaced the other. The unresolved question is how the system should decide that one memory supersedes another when both are individually relevant. A stricter system could require explicit evidence that a decision was changed, while a more flexible system could infer changes from later conversations but the first may fail to recognize genuine updates, while the second may incorrectly mark valid information as obsolete.

5) **Memory write decision:** We identified the need for write-side judgment, but haven't decided how much evidence should be required before something becomes a durable memory. Storing almost everything increases recall but creates noise and future conflicts, being more selective keeps the memory cleaner but risks permanently losing information that later turns out to be important.